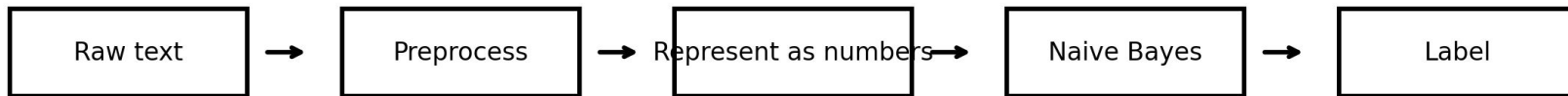


Bayes Classification

ICT 3.3 Machine learning and Data Mining



Text Classification



A classifier cannot read raw text directly: we build a numerical representation first.

Text Classification

- Spam / Not spam email
- Happy / Not Happy hotel review
- Sports / Politics / Technology article
- Positive / Negative tourist comment

raw text → **tokens** → **numbers**
→ **label**

```
"The room was dirty but the staff  
were nice."  
→ [room, dirty, staff, nice]  
→ [nice:1, dirty:1, great:0, ...]  
→ Not Happy
```

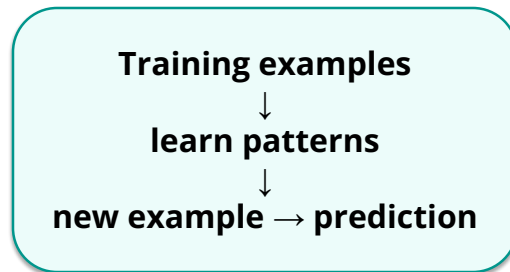
Text classification is one of the simplest and most useful NLP tasks.

Why Classification?

Humans and machines learn categories from examples

We classify because we cannot treat every object as completely unique.

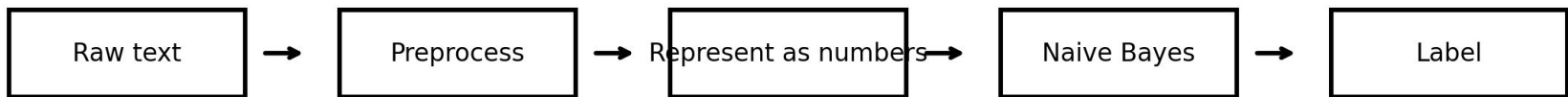
- Human examples: cat vs dog, irony vs sincere, familiar voice vs unknown voice
- Machine examples: spam detection, sentiment analysis, topic classification
- A model learns patterns from labeled examples, then predicts labels for new examples



"great hotel" → Happy
"dirty room" → Not Happy
"great staff, dirty room" →
?

NLP: messy text → usable data

Raw text is written for humans. Machine learning models need regular, numerical input.



A classifier cannot read raw text directly: we build a numerical representation first.

We first transform it into features.

Tokenization

Tokenize on rules	Let	's	tokenize	!	Is	n't	this	easy	?		
Tokenize on punctuation	Let	'	s	tokenize	!	Isn	'	t	this	easy	?
Tokenize on white spaces	Let's	tokenize!	Isn't	this	easy?						

Let's tokenize! Isn't this easy?

Tokens can be words, punctuation, characters, or subwords. The choice affects what the model can learn.

N-grams

An n-gram is a sequence of N words.

1-Gram	2-Gram	3-Gram
The	The Margherita	The Margherita pizza
Margherita	Margherita pizza	Margherita pizza is
pizza	pizza is	pizza is not
is	is not	is not bad
not	not bad	not bad taste
bad	bad taste	
taste		

“good” and “not good” should not mean the same thing.

Preprocessing

Make the text simpler and more regular:

- **Lowercase:** Hotel → hotel
- **Remove punctuation:** hotel! → hotel
- **Remove stopwords:** the, and, is, of

Raw review:

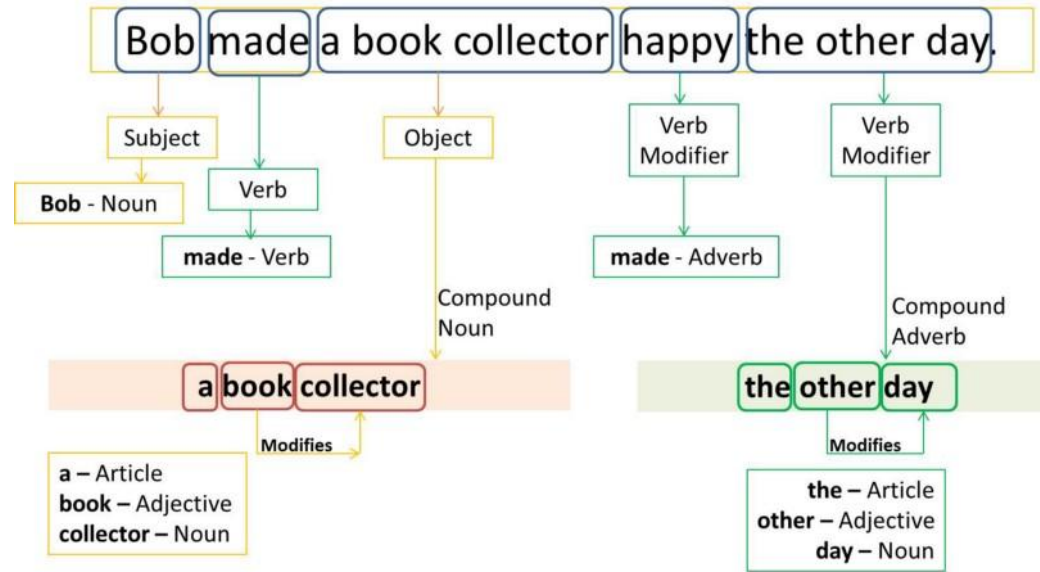
```
"Stayed here with my sons. We  
loved the hotel!"
```

Clean tokens:

```
stay, sons, love, hotel
```

Preprocessing

- **Lemmatization:** loved → love, better → good
- **Stemming:** programming → program
- **Part-of-Speech:** verbs, nouns..



Preprocessing

- **Lowercase:** Hotel → hotel
- **Remove punctuation:** hotel! → hotel
- **Remove stopwords:** the, and, is, of
- **Lemmatization:** loved → love, better → good
- **Stemming:** programming → program
- **Part-of-Speech:** verbs, nouns..

**Important: preprocessing choices
change the evidence available to the
classifier.**

Text representation

A classifier needs numbers, not raw sentences

Common representations:

- **One-hot** encoding: one vector per word
- **Bag-of-words**: count words in a document
- **TF-IDF**: count words, but downweight very common words
- **Embeddings**: represent meaning with dense vectors

Human-Readable

Pet
Cat
Dog
Turtle
Fish
Cat



Machine-Readable

Cat	Dog	Turtle	Fish
1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1
1	0	0	0

One Hot Encoding

The representation is one of the simplest representations because all the words are independent from each other.

The size of the representation increases with the number of the total words in a dataset.

"The hotel was OK.

The room was clean."

[etc]

→ vocabulary = 6 unique words

Imagine that we have a vocabulary of 50,000. (There are about a million words in English.)

Each word is represented by 49,999 zeros and a single 1.

→ we need $50.000^2 = 2.5$ billion units of memory space.

Not efficient in terms of computation.

the	hotel	was	OK	room	clean
1	0	0	0	0	0
0	1	0	0	0	0
0	0	1	0	0	0
0	0	0	1	0	0
0	0	0	0	1	0
0	0	0	0	0	1

Bag-of-Words

The Bag of Words Representation

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!

15



it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
...	...

Bag-of-Words

Differently from the **one hot encoding** representation, the **bag-of-words** is a representation of text that describes the occurrence of words within a document. It is called a “*bag*” of words, because any information about the order or structure of words in the document is discarded. The model is only concerned with whether known words occur in the document, not where in the document.

1. *“The hotel was OK, very very clean.”*
2. *“The room was clean.”*

→ 2 documents, vocabulary = 7 unique words

	the	hotel	was	OK	very	clean	room
1	1	1	1	1	2	1	0
2	1	0	1	0	0	1	1

```
text → x
x = [count(great),
     count(nice), count(dirty),
     count(noise), ...]
```

14

This is why Naive Bayes is often introduced with text classification.

Bag-of-Words

It is called a “*bag*” of words, because any information about the order or structure of words in the document is discarded. The model is only concerned with **whether known words occur in the document, not where in the document.**

Why is the order or the position of a word in a text important?

Bag-of-Words

Sentence 1:

“The dog bites the man.”

Sentence 2:

“The man bites the dog.”

Bag-of-Words

Sentence 1:

“The dog bites the man.”

Sentence 2:

“The man bites the dog.”

Bag-of-Words Representation

dog = 1 bites = 1 man = 1

→ Same vector, but different meaning

17

Bag-of-Words ignores word order. It only counts which words appear, not where they appear.

Raw counting problem: “empty” words are the most frequent

“and”, “the”, etc.

charles bailey WAS indicted for feloniously stealing on **the** 29th of december two dressed deer skins value 20 s **the** property **of** samuel savage **and** richard savage richard savage i am a leather seller 63 chinwell street my partner s name is samuel savage a few days previous to **the** 29th of december i looked out seventy skins for an order these skins being **of** a bad colour i directed them to be brimstoned to make them **of** equal colour pale on **the** 29th in **the** afternoon i saw them all smooth on a horse a few hours afterwards they appeared very much tumbled **and** one WAS thrown into **the** yard **and** dirtied i caused them to be brought in **the** warehouse **and** counted there WAS two gone our foreman went to worship street **and** brought armstrong **and** vickrey they searched **and** found this skin in **the** prisoner s breeches **and** **the** other skin was found in **the** workshop carter i am foreman to samuel **and** richard savage **the** seventy skins i was with mr savage looking them out i took them out of **the** stove **and** counted them on **the** horse **and** on friday i counted them three times over there were no more than sixty eight instead **of** seventy i went to worship street brought mr armstrong **and** vickrey with me they waited till **the** men left work **and** when they came down they were searched **and** on **the** prisoner one skin was found john armstrong i went to this gentleman s home after **the** men came down vickrey **and** i were searching in one minute vickrey called me i received this skin from him it WAS taken out of **the** prisoner s breeches i have had it ever since john vickrey q you were with armstrong

TF-IDF (Term Frequency-Inverse Document Frequency)

The **TF-IDF** (Term Frequency-Inverse Document Frequency) statistical measure makes it possible to evaluate the importance of a term contained in a document, relative to a collection or a corpus. The weight increases in proportion to the number of occurrences of the word in the document. It also varies according to the frequency of the word in the corpus.

TF = (#Number of repetitions of a word in a document) / (#Number of words in a document)

IDF = $\text{Log}[(\# \text{Number of documents}) / (\# \text{Number of documents containing the word})]$

1. *"The hotel was OK, very very clean."* → vocabulary = 7 total words

"the" → TF = $1/7 = 0,14$

"hotel" → TF = $1/7 = 0,14$

"was" → TF = $1/7 = 0,14$

"OK" → TF = $1/7 = 0,14$

"very" → TF = $2/7 = 0,28$

"clean" → TF = $1/7 = 0,14$

"the" → IDF = $\log(2/2) = 0$

"hotel" → IDF = $\log(2/1) = 0,30$

"was" → IDF = $\log(2/2) = 0$

"OK" → IDF = $\log(2/1) = 0,30$

"very" → IDF = $\log(2/1) = 0,30$

"clean" → IDF = $\log(2/2) = 0$

"the" → TF*IDF = 0

"hotel" → TF*IDF = $0,14*0,30 = 0,042$

"was" → TF*IDF = 0

"OK" → TF*IDF = $0,14*0,30 = 0,042$

"very" → TF*IDF = $0,28*0,30 = 0,084$

"clean" → TF*IDF = 0

TFIDF

2. *"The room was clean."* → 2 documents, vocabulary = 4 total words

"the" → TF = $1/4 = 0,25$

"hotel" → TF = $1/4 = 0,25$

"was" → TF = $1/4 = 0,25$

"clean" → TF = $1/4 = 0,25$

.....

For a term i in document j :

$$w_{i,j} = tf_{i,j} \times \log\left(\frac{N}{df_i}\right)$$

$tf_{i,j}$ = number of occurrences of i in j
 df_i = number of documents containing i
 N = total number of documents

Embeddings

Problems:

- Vector size is too large
- Vocabulary is too large
- Vectors know nothing about **meaning**

e.g., *cat* is as close to *dog* as it is to *table*!

What is **meaning**?



Embeddings

How to represent the similarities of words?

The altitude of Mont Blanc is 4,810 meters.
The height of Mont Blanc is 4,810 meters.



Embeddings

How to represent the similarities of words?

The altitude of Mont Blanc is 4,810 meters.

The height of Mont Blanc is 4,810 meters.

One-Hot Vectors

altitude = [1, 0, 0]

height = [0, 1, 0]

table = [0, 0, 1]

...

Euclidean Distances

Distance(altitude, height) = $\sqrt{2} = 1.41$

Distance(altitude, table) = $\sqrt{2} = 1.41$



Embeddings

How to represent the similarities of words?

The altitude of Mont Blanc is 4,810 meters.

The height of Mont Blanc is 4,810 meters.

In a classic vector representation (one-hot encoding ...):

Distance (*altitude*, *height*) = Distance (*altitude*, *table*)

No consideration of semantic proximity! **Meaning!**



Embeddings

What is semantic meaning?

What is Semantic Meaning?

altitude ↔ **height**

→ similar meaning

car ↔ **automobile**

→ same concept

doctor ↔ **hospital**

→ related concepts

apple ↔ **banana**

→ same category

Semantic meaning means:

- Words can be close in meaning
- Words can appear in similar contexts
- Different words can express similar ideas
- Meaning matters, not only spelling

Humans understand that "altitude" and "height" are semantically similar.
Modern NLP models try to learn this semantic meaning automatically.

Embeddings

How to learn the meaning of words?

1. Word embeddings!
2. Contextual embeddings!

26

Firth (1957): *"You shall know a word by the company it keeps!"*

→ **Learning the meaning of a word through its contexts of use**



John Rupert Firth (1890-1960) was an English linguist and a leading researcher in British linguistics in the 1950s.

Embeddings

“Tesgüino” ?



1. Finish lake



2. Mexican drink



3. Japanese manga

Embeddings

Tesgüino ?

A bottle of tesgüino is on the table.

Tesgüino is produced in the Sierra Madre Occidental region of Mexico.

Drinking tesgüino makes you drunk.

Tesgüino is made from corn.

Embeddings

Tesgüino ?

A bottle of tesgüino is on the table.

Tesgüino is produced in the Sierra Madre Occidental region of Mexico.

Drinking tesgüino makes you drunk.

Tesgüino is made from corn.



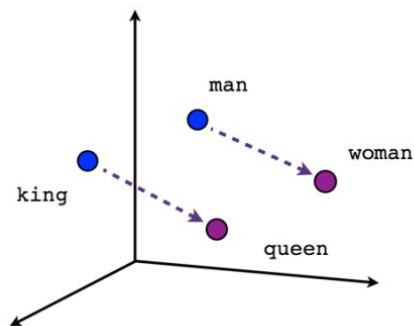
→ **Learning the meaning of a word through its contexts of use**

Embeddings

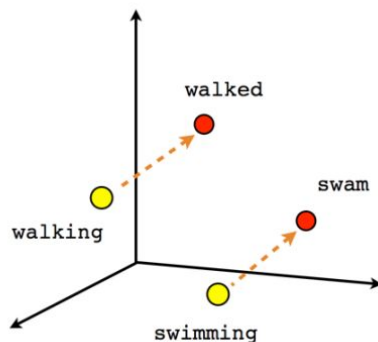
Word embeddings are a type of word representation that allows words with **similar meaning** to have a **similar representation**. Words or phrases from the vocabulary are mapped to vectors of **real numbers**.

Conceptually, it involves a mathematical embedding from space with many dimensions per word to a continuous vector space with a much lower dimension. The representations are usually generated by neural networks.

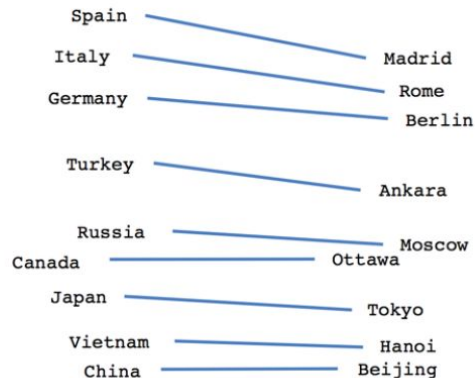
word = [0.004, -0.88, 3.01, ...etc]



Male-Female



Verb tense

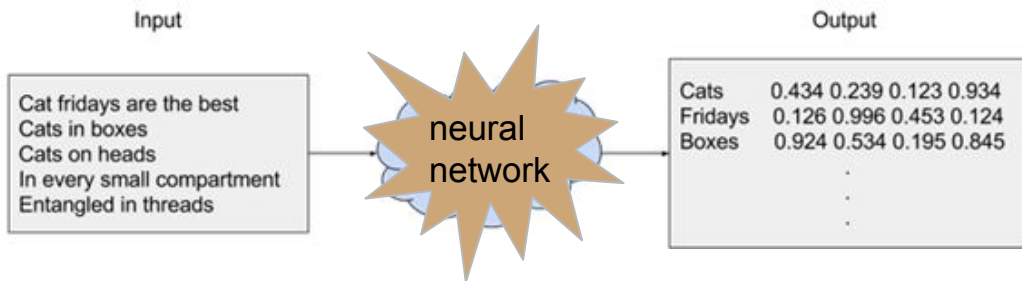


Country-Capital

Embeddings

- This representation makes it possible to represent each word of a dictionary by a vector of **real numbers**.
- Words with similar contexts have vectors that are relatively close.
- This technique is based on the assumption that words appearing in similar contexts have related meanings:
 - “dog” and “cat” (pets)
 - “Saturday” and “Sunday” (days in a week)
 - “red wine”, “beer” (alcoholic beverages)

31



Embeddings

all vocabulary words $|V|=50,000$

probabilities $\longrightarrow \dots p(\text{another word}|\text{mat}) \ p(\text{the}|\text{mat}) \ p(\text{another word}|\text{mat})\dots$

activation function for
normalizing the output of a
network to a probability
distribution over predicted
output classes

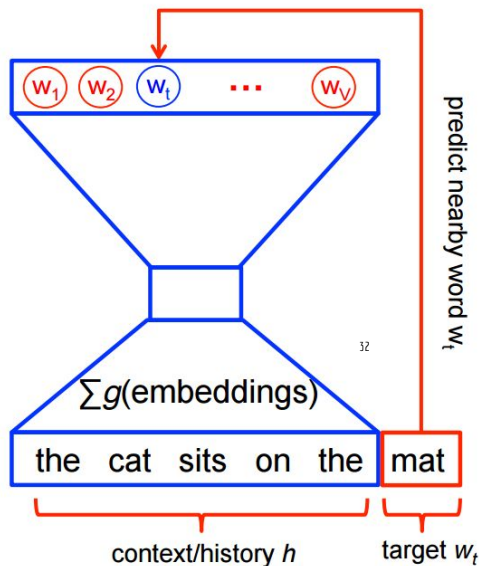
$\longrightarrow$ Softmax classifier

most calculations are here
 $W \times V_{\text{the}} + b$

$\longrightarrow$ Hidden layer

$\longrightarrow$ Projection layer

0 4 9 7 867 67
The cat sits on the mat



vocabulary $|V|=50,000$

0	1	2	3	4	5	..	50,000
the	a	in	with	cat	dog	..	

the	→	0.34	0.4	0.11	0.5	0.89
cat	→	0.6	0.23	0.8	0.87	0.21
dog	→	0.77	0.21	0.09	0.29	0.05
...						

A Quick Reminder on Meaning

In one-hot or bag-of-words representations, “cat” and “dog” are not naturally closer than “cat” and “table”. Embeddings try to solve this by placing words used in similar contexts near each other.

```
hotel, room, breakfast,  
    reception  
cat, dog, pet, animal  
bank near river ≠ bank for  
    deposit
```

→ we stay with **bag-of-words** because it makes Bayes easy to understand.

Naive Bayes Classification

Training data contains inputs with known labels.

"great experience, delicious dinner" → Happy
"not a nice room, miserable stay" → Not
Happy
"nice staff, helpful service" → Happy

Naive Bayes learns two things:

- How common each class is: $P(y)$
- Which words are common in each class: $P(\text{word} \mid y)$

**Then for a new review, it asks:
Which class makes these words more likely?**

Naïve Bayes

For each possible class, compute a score. Choose the class with the highest score.

$$\hat{y} = \arg \max_y P(y \mid x)$$

x = features, for example the words in a review

y = class, for example Happy or Not Happy

$\hat{y}$ = predicted class

Naïve Bayes

For each possible class, compute a score. Choose the class with the highest score.

$$\hat{y} = \arg \max_y P(y \mid x)$$

choose the class y that has the highest probability after seeing x .

x = features, for example the words in a review

y = class, for example Happy or Not Happy

$\hat{y}$ = predicted class

“Given these words ‘ x ’, which label ‘ y ’ is most believable?”

Bayes Rule

$$P(y | x) = \frac{P(x | y)P(y)}{P(x)}$$

$P(y | x)$
posterior →
what we want

$P(x | y)$
likelihood →
how well class explains x

$P(y)$
prior →
how common class is

$P(x)$
evidence →
how common x is overall

Bayes combines what we already know about the class with how well the class explains the observed data.

Comparing Classes/Labels

Same denominator

$$P(y | x) = \frac{P(x | y)P(y)}{P(x)}$$

$P(x)$ is the same for Happy and Not Happy, so it does not change which class wins.

$$\hat{y} = \arg \max_y P(y | x)$$

We compare: likelihood $\times$ prior

The “Naive” Assumption

Without simplification, estimating $P(\text{all words} \mid \text{class}) = P(x \mid y)$ is hard.

$$P(x_1, x_2, x_3 \mid y) = P(x_1 \mid y) P(x_2 \mid y) P(x_3 \mid y)$$

$$\begin{aligned} & P(\text{nice, great, dirty} \mid \text{Happy}) \\ &= P(\text{nice} \mid \text{Happy}) \times P(\text{great} \mid \text{Happy}) \times P(\text{dirty} \mid \text{Happy}) \end{aligned}$$

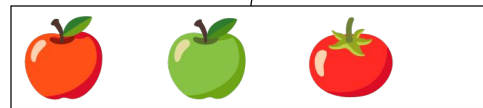
Naive means “we simplify the world so the model can learn from limited data.”

Bayes Classification

$P(\text{Apple}) = 0.7$ because 7 out of 10 fruits are apples.

$P(\text{Tomato}) = 0.3$ because 3 out of 10 are tomatoes.

Dataset: 10 examples



...

Bayes Classification

$P(\text{Apple}) = 0.7$ because 7 out of 10 fruits are apples.

$P(\text{Tomato}) = 0.3$ because 3 out of 10 are tomatoes.

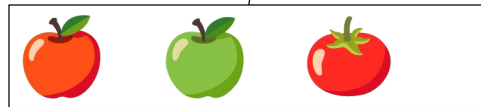
$P(\text{Red} \mid \text{Apple}) = 0.4$ only 40% of apples are red.

$P(\text{Red} \mid \text{Tomato}) = 0.9$ 90% of tomatoes are red.

evidence

red

Dataset: 10 examples



...

Bayes Classification

$P(\text{Apple}) = 0.7$ because 7 out of 10 fruits are apples.

$P(\text{Tomato}) = 0.3$ because 3 out of 10 are tomatoes.

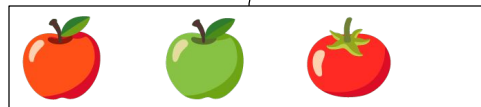
$P(\text{Red} \mid \text{Apple}) = 0.4$ only 40% of apples are red.

$P(\text{Red} \mid \text{Tomato}) = 0.9$ 90% of tomatoes are red.

evidence

red

Dataset: 10 examples



...

Tomato score = $P(\text{Red} \mid \text{Tomato})P(\text{Tomato}) = 0.9 \times 0.3 = 0.27$

Apple score = $P(\text{Red} \mid \text{Apple})P(\text{Apple}) = 0.4 \times 0.7 = 0.28$

Bayes Classification

$P(\text{Apple}) = 0.7$ because 7 out of 10 fruits are apples.

$P(\text{Tomato}) = 0.3$ because 3 out of 10 are tomatoes.

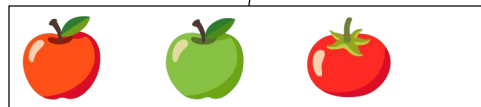
$P(\text{Red} \mid \text{Apple}) = 0.4$ only 40% of apples are red.

$P(\text{Red} \mid \text{Tomato}) = 0.9$ 90% of tomatoes are red.

evidence

red

Dataset: 10 examples



...

Tomato score = $P(\text{Red} \mid \text{Tomato})P(\text{Tomato}) = 0.9 \times 0.3 = 0.27$

Apple score = $P(\text{Red} \mid \text{Apple})P(\text{Apple}) = 0.4 \times 0.7 = 0.28$

$\hat{y} = \text{Apple}$

Bayes Classification

Red is more typical for tomatoes

BUT

Apples are much more common overall

$\hat{y} = \text{Apple}$ 

Bayes classifier does not only ask: *"Is red typical for tomatoes?"*

It asks: *"Considering both how common the class is and how typical the evidence is, which class is more likely?"*

Bayes Classification

Evidence accumulates feature by feature → **Example: apple vs tomato**

Features: red + round + about 10 cm
(evidence)



$$\begin{aligned} P(\text{Apple} \mid x) &\propto P(\text{Apple}) \\ &\times P(\text{red} \mid \text{Apple}) \\ &\times P(\text{round} \mid \text{Apple}) \\ &\times P(10\text{cm} \mid \text{Apple}) \end{aligned}$$

The model treats color, shape, and size as separate pieces of evidence.

Text Example: Word Evidence

Training data: 6 Happy reviews and 3 Not Happy reviews.

Happy:
great=6, nice=2, helpful=3, dirty=0
→ 11 words

Not Happy:
great=1, nice=1, helpful=0, dirty=2
→ 4 words

$$P(\text{great} \mid \text{Happy}) = 6 / 11 = 0.54$$

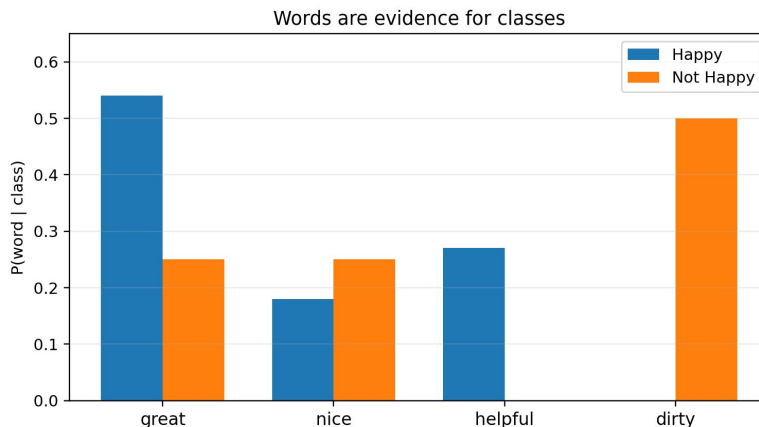
$$P(\text{nice} \mid \text{Happy}) = 2 / 11 = 0.18$$

$$P(\text{helpful} \mid \text{Happy}) = 3 / 11 = 0.27$$

$$P(\text{great} \mid \text{Not Happy}) = 1 / 4 = 0.25$$

$$P(\text{nice} \mid \text{Not Happy}) = 1 / 4 = 0.25$$

$$P(\text{helpful} \mid \text{Not Happy}) = 0 / 4 = 0$$



Classifying a New Review

New review:
"The staff were great and very helpful."

Use the words: great, helpful

$$\begin{aligned} \text{score(Happy)} &= \\ P(\text{Happy}) \times P(\text{great}|\text{Happy}) \times P(\text{helpful}|\text{Happy}) &= 0.66 \times 0.54 \times 0.27 = 0.096 \end{aligned}$$

$$\begin{aligned} \text{score(Not Happy)} &= \\ P(\text{Not Happy}) \times P(\text{great}|\text{Not Happy}) \times P(\text{helpful}|\text{Not Happy}) &= 0.33 \times 0.25 \times 0 = 0 \end{aligned}$$

Prediction: Happy

The Zero Problem

If one word never appeared with a class, its probability is 0.

$$P(\text{helpful} \mid \text{Unhappy}) = 0 \rightarrow \text{score}(\text{Unhappy}) = 0$$

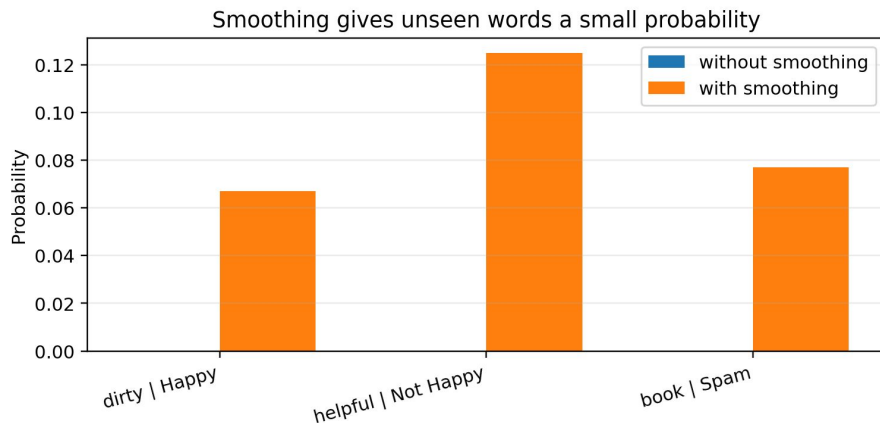
But maybe “dirty” simply never appeared in the small Happy training set.

Solution: smoothing

$$P(w \mid y) = (\text{count}(w, y) + \alpha) / (\text{total_words}(y) + \alpha |V|)$$

With $\alpha = 1$, unseen words get a small probability instead of zero. V = size of vocab

Smoothing



Smoothing prevents the model from becoming too certain because of missing evidence.

- Without smoothing: unseen word $\rightarrow$ probability 0
- With smoothing: unseen word $\rightarrow$ small probability
- The prediction may remain the same, but the calculation is safer

Log Probabilities

In practice, we add instead of multiply. Products of many small probabilities become extremely tiny.

$$\text{score}(y) = \log P(y) + \sum_{w \in x} \text{count}(w, x) \cdot \log P(w \mid y)$$

Same decision, easier computation

multiply probabilities → add
log-probabilities

This is one reason Naive Bayes is fast even with many words (features/evidence).

Log Probabilities

In practice, we add instead of multiply. Products of many small probabilities become extremely tiny.

$$\text{score}(y) = \log P(y) + \sum_{w \in x} \text{count}(w, x) \cdot \log P(w \mid y)$$

```
score(Happy)
= P(Happy) × P(great | Happy) ×
P(great | Happy) × P(nice | Happy)
```

Same decision, easier computation

multiply probabilities → add
log-probabilities

This is one reason Naive Bayes is fast even with many words (features/evidence).

Why It Works Surprisingly Well

Naive Bayes Assumption

- Words are assumed independent given the class.
- Example: "free" and "money" are treated separately.
- This is not fully true in real language.

Naive ≠ stupid

"Naive" means we simplify the problem so the model can estimate probabilities from limited data.

Why It Still Works

- Text contains many useful weak signals.
- Words like "dirty", "excellent", "refund", "free" carry strong evidence.
- The model is simple, fast, and interpretable.

$$\begin{aligned} &P(\text{free, money} \mid \text{Spam}) \\ &\approx P(\text{free} \mid \text{Spam}) \times P(\text{money} \mid \text{Spam}) \end{aligned}$$

Why It Works Surprisingly Well

Naive Bayes Assumption

- Words are assumed independent given the class.
- Example: "free" and "money" are treated separately.
- This is not fully true in real language.

Naive ≠ stupid

"Naive" means we simplify the problem so the model can estimate probabilities from limited data.

Why It Still Works

- Text contains many useful weak signals.
- Words like "dirty", "excellent", "refund", "free" carry strong evidence.
- The model is simple, fast, and interpretable.

$$\begin{aligned} &P(\text{free, money} \mid \text{Spam}) \\ &\approx P(\text{free} \mid \text{Spam}) \times P(\text{money} \mid \text{Spam}) \end{aligned}$$

<https://www.interactive-ml.com/naive-bayes.html>

Interactive Naive Bayes Algorithm

Learn how Naive Bayes classification works through interactive examples

What is Naive Bayes?

Naive Bayes is a probabilistic machine learning algorithm based on Bayes' theorem. It's called "naive" because it assumes that all features are independent of each other given the class label - which is rarely true in real life, but often works surprisingly well in practice!

 **The Goal:** Given some features (like words in an email or weather conditions), predict which class an example belongs to (like spam/ham or play/don't play tennis).

Bayes' Theorem

$$P(\text{Class} \mid \text{Features}) = P(\text{Features} \mid \text{Class}) \times P(\text{Class}) / P(\text{Features})$$

In plain English: "The probability that this belongs to a class, given the features we observe, equals the probability of seeing these features in that class, times how common that class is, divided by how common these features are overall."

Key Components

Prior $P(\text{Class})$

How common is each class in our training data? If 40% of emails are spam, then $P(\text{Spam}) = 0.4$

Likelihood $P(\text{Feature} \mid \text{Class})$

How often does each feature appear in each class?
How often does "free" appear in spam vs. ham emails?

Posterior $P(\text{Class} \mid \text{Features})$

Our final answer! The probability that an example belongs to each class given the features we observed.

Mini Assignment

TODOs:

- Exercise notebook

Clone the repo!



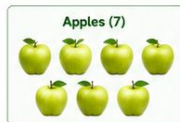
Understanding Precision, Recall, Accuracy and F1 Score

Example: Detecting Apples vs Tomatoes

1. The Real World (Ground Truth)

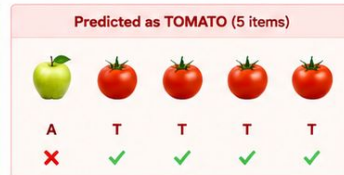
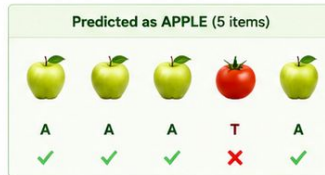
There are 10 fruits in total:

- 7 Apples (Positive class)
- 3 Tomatoes (Negative class)



2. Model Predictions

The model predicts 5 items as Apples and 5 items as Tomatoes.



What does this mean?

- ✓ Correct prediction
- ✗ Incorrect prediction

3. Confusion Matrix (Actual vs Predicted)

		ACTUAL (Ground Truth)	
		APPLE (Positive) 7 items	TOMATO (Negative) 3 items
PREDICTED by MODEL	Predicted APPLE (Positive)	TRUE POSITIVE (TP) 4 Actual Apple predicted as Apple	FALSE POSITIVE (FP) 1 Actual Tomato predicted as Apple
	Predicted TOMATO (Negative)	FALSE NEGATIVE (FN) 3 Actual Apple predicted as Tomato	TRUE NEGATIVE (TN) 2 Actual Tomato predicted as Tomato

TP (True Positive):
We predicted Apple and it was actually an Apple.

FP (False Positive):
We predicted Apple but it was actually a Tomato.

FN (False Negative):
We predicted Tomato but it was actually an Apple.

TN (True Negative):
We predicted Tomato and it was actually a Tomato.

4. Counts Summary

TP (True Positives)	= 4
FP (False Positives)	= 1
FN (False Negatives)	= 3
TN (True Negatives)	= 2
Total items	= 10
Actual Apples	= 7
Actual Tomatoes	= 3
Predicted Apples	= 5
Predicted Tomatoes	= 5

5. Metrics Calculation

Precision

Out of all items predicted as Apple, how many are actually Apples?

$$\text{Precision} = \frac{TP}{TP + FP}$$
$$= \frac{4}{4 + 1} = \frac{4}{5}$$

= 0.80 (80%)

Recall (Sensitivity)

Out of all actual Apples, how many did we detect?

$$\text{Recall} = \frac{TP}{TP + FN}$$
$$= \frac{4}{4 + 3} = \frac{4}{7}$$

= 0.571 (57.14%)

Accuracy

Out of all items, how many did we predict correctly?

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}$$
$$= \frac{4 + 2}{4 + 1 + 3 + 2} = \frac{6}{10}$$

= 0.60 (60%)

F1 Score

Harmonic mean of Precision and Recall

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$
$$= 2 \times \frac{0.80 \times 0.571}{0.80 + 0.571}$$

= 0.667 (66.67%)

6. Visual Summary of Errors

- ✓ 4 **Correctly detected Apples (TP)**
Good! Model found these apples.
- ✗ 3 **Missed Apples (FN)**
These apples were missed (predicted as Tomato).
- ✗ 1 **Tomatoes wrongly predicted as Apple (FP)**
This tomato was a false alarm.
- ✓ 2 **Correctly rejected Tomatoes (TN)**
Good! Model rejected these tomatoes.



Key Takeaway: High precision means few false alarms (tomatoes predicted as apples). High recall means few missed apples. Accuracy tells overall correctness. F1 balances precision and recall.